

Trabalho Prático 2: Problemas de Roteamento (PCV e Caminho Hamiltoniano) com Análise de Complexidade e Green Computing

Curso: Ciência da Computação - UNIVALI

Disciplina: Teoria da Computação

Objetivo geral: desenvolver uma solução computacional para um problema prático análogo ao Problema do Caixeiro Viajante (PCV/TSP) ou Caminho Hamiltoniano, investigando abordagens algorítmicas, analisando formalmente sua complexidade e discutindo o impacto da eficiência do algoritmo em termos de green computing.

Temas disponíveis (escolha de um)

Cada equipe (3 alunos) deve escolher **um** dos cinco temas abaixo, todos modeláveis como TSP (ciclo mínimo) ou Caminho Hamiltoniano (caminho que visita cada vértice uma única vez).

1. Roteirização e entrega com múltiplas restrições de aresta (PCV enriquecido)

- Contexto: uma frota (pode ser reduzida a um veículo para simplificar) realiza entregas em diferentes clientes de uma cidade.
- Cada aresta (ligação entre dois pontos) possui um custo composto, por exemplo: distância física, tempo de deslocamento, pedágios, consumo estimado de combustível, emissão de CO₂, risco de trânsito, etc.
- Tarefa: modelar o problema como grafo ponderado com pesos multicritério, definir uma função de custo agregada (por exemplo combinação linear dos critérios) e encontrar um ciclo (PCV) de menor custo que visite todos os pontos.
- Extensão opcional: comparar uma rota “ingênua” (por exemplo, sempre escolher a aresta mais curta) com uma rota obtida por heurística mais elaborada, medindo impacto em número de operações e custo energético estimado.

2. Rota de envio/recebimento de pacotes em redes TCP/IP

- Contexto: modelar uma sub-rede de Internet (roteadores, enlaces, latências, perdas) e definir uma rota de passagem de um fluxo de pacotes que precisa visitar uma sequência de nós (por exemplo, proxies, firewalls, servidores de cache) antes de chegar ao destino.
- Cada aresta representa um enlace de rede e possui custo que pode combinar latência, vazão disponível, taxa de perda, número de saltos, e consumo energético estimado por pacote.

- Tarefa: formular o problema como TSP ou Caminho Hamiltoniano (por exemplo, encontrar uma rota que visite todos os nós de inspeção ou caching uma única vez com menor custo agregado) e projetar um algoritmo/heurística de roteamento.
- Extensão opcional: discutir relação com protocolos reais (ex.: roteamento baseado em métricas, balanceamento de carga, green networking).

3. Otimização de robôs de perfuração de placas de circuito impresso (PCB drilling)

- Contexto: um braço robótico precisa perfurar uma lista de furos em uma placa de circuito impresso, minimizando o deslocamento total para reduzir tempo, desgaste mecânico e consumo de energia.
- Cada furo é um vértice; o custo entre dois furos pode incluir distância geométrica, tempo de aceleração/frenagem do braço, energia consumida no movimento, e possíveis restrições de ordem (por exemplo, zonas que devem ser percorridas antes de outras).
- Tarefa: modelar o problema como TSP (ciclo que parte e retorna a uma posição de repouso) ou Caminho Hamiltoniano (somente ida) e implementar um algoritmo capaz de gerar uma ordem de perfuração de custo mínimo ou aproximado.
- Extensão opcional: simular diferentes estratégias (ex.: vizinho mais próximo, 2-opt, algoritmos genéticos) e comparar desempenho e impacto energético.

4. Otimização da ordem de limpeza de vidraças em fachadas de prédios

- Contexto: um robô ou equipe de limpeza deve limpar todas as janelas de um prédio (dispostas em linhas e colunas), minimizando movimentos verticais e horizontais, tempo total e gasto energético do equipamento.
- Cada janela é um vértice; o custo de deslocamento entre janelas pode considerar a distância, o tempo de reposicionamento da plataforma, o esforço para subir/descer, e o consumo de energia.
- Tarefa: representar a fachada como um grafo (por exemplo, grade 2D), formular o problema como TSP (rota fechada) ou Caminho Hamiltoniano (rota aberta) e propor um algoritmo que determine a ordem de limpeza menos custosa.
- Extensão opcional: analisar padrões de varredura (linha a linha, serpentina, zigue-zague) comparando complexidade e estimativa de consumo energético.

5. Otimização da inspeção de fuselagem de aeronaves

- Contexto: robôs automatizados percorrem a fuselagem de um avião para inspecionar milhares de rebites em busca de rachaduras ou falhas, e o objetivo é minimizar a distância percorrida e o tempo de inspeção.
- Cada ponto de inspeção (rebite) é um vértice; o custo entre pontos considera distância ao longo da superfície, tempo de movimento, energia gasta e possivelmente áreas de difícil acesso.
- Tarefa: modelar o problema como TSP sobre a superfície da aeronave e implementar uma abordagem que gere uma sequência de inspeção de custo mínimo ou aproximado.
- Extensão opcional: discutir como a melhoria da rota impacta diretamente o tempo de liberação da aeronave e o consumo energético em hangar, iluminação e operação dos robôs.

Requisitos técnicos da implementação

Mantêm-se as características gerais do Trabalho 1.

- Linguagem de programação: livre escolha da equipe.
- Instâncias do problema: o programa deve permitir configurar o tamanho do problema (número de vértices/pontos) e, quando fizer sentido, gerar instâncias aleatórias e/ou carregar instâncias de arquivo.
- Parametrização:
 - Escolha do tema e do tipo de problema (PCV ou Caminho Hamiltoniano).
 - Definição de parâmetros de custo (pesos dos critérios na função de custo agregada).
- Visualização: exibir de maneira compreensível:
 - a sequência de vértices na rota;
 - o custo total da solução;
 - métricas auxiliares (por exemplo: distância total percorrida, número de operações aproximado, uso de memória estimado).

Atividades obrigatórias das equipes

Cada equipe deve realizar, documentar e apresentar as seguintes etapas.

1. Escolha e contextualização do tema

- Descrever claramente o cenário real (entregas, rede TCP/IP, robô de perfuração, limpeza de vidraças ou inspeção de fuselagem).
- Identificar por que o problema é análogo a PCV ou Caminho Hamiltoniano.

2. Pesquisa da abordagem de solução

- Levantar pelo menos duas abordagens: exaustiva/ingênua (backtracking, força bruta) e uma abordagem mais eficiente ou heurística (algoritmos gulosos, metaheurísticas, etc.).
- Referenciar materiais utilizados (artigos, livros, sites), incluindo pelo menos uma referência de green computing ou eficiência de algoritmos.

3. Modelagem formal do problema

- Definir o grafo: conjunto de vértices, arestas, função de custo, restrições.
- Especificar claramente se o objetivo é encontrar um ciclo hamiltoniano mínimo (TSP) ou um caminho hamiltoniano mínimo.
- Documentar a função de custo (como os múltiplos critérios são agregados e por que).

4. Implementação da solução

- Implementar pelo menos uma abordagem que seja capaz de resolver instâncias pequenas a médias, de forma reproduzível.
- Opcional (fortemente recomendado): implementar duas abordagens para fins de comparação (por exemplo, força bruta vs heurística).

5. Análise de complexidade

- Determinar a complexidade de tempo e espaço da(s) abordagem(ns) implementada(s), em notação Big-O.
- Relacionar a complexidade teórica com medições empíricas: tempo de execução aproximado e uso de memória para diferentes tamanhos de entrada quantidade de operações necessárias.

6. Análise de adequação em Green Computing

- Explicar como a eficiência algorítmica impacta o consumo de energia, citando ideias como: menos operações, menor tempo de

CPU, menos acesso a disco/rede, menor consumo em data centers.

- Discutir explicitamente como a escolha de algoritmos e estruturas de dados mais eficientes pode reduzir o consumo energético e a pegada de carbono, referenciando os textos fornecidos (“Impact of Algorithms on Green Computing” e “Green Algorithms The Environmental Cost of Code”).
- Opcional: propor métricas simples de “custo energético estimado” (por exemplo, associar um valor de energia por operação ou por segundo de CPU e comparar soluções).

Entregáveis

1. Código-fonte

- Completo, organizado e com instruções claras de compilação/execução.
- Comentários suficientes para entender as principais funções e estruturas, especialmente onde houver escolhas relacionadas à eficiência/green computing.

2. Relatório técnico (em formato acadêmico)

O relatório deve conter, no mínimo:

- Introdução ao tema escolhido e motivação prática.
- Modelagem do problema (definição do grafo, tipo de problema, função de custo).
- Descrição das abordagens algorítmicas estudadas, com referências.
- Detalhamento da implementação escolhida.
- Análise de complexidade (tempo e espaço) com notação Big-O.
- Resultados experimentais:
 - instâncias testadas;
 - tempos de execução;
 - comportamento ao aumentar o tamanho do problema;
- Se possível, comparação entre duas abordagens (ingênua vs otimizada/heurística).
- Seção específica “Discussão de Green Computing”:

- síntese dos conceitos principais dos artigos fornecidos (sem copiar trechos);
- relação entre eficiência algorítmica e consumo de energia no cenário estudado;
- justificativa de porque a abordagem escolhida pode ser considerada mais ou menos adequada sob a perspectiva de green computing.
- Conclusões e possíveis extensões futuras.

3. Apresentação oral

- Cada equipe apresentará seu trabalho para a turma e professor.
- Todos os integrantes devem demonstrar entendimento do problema, da modelagem, do algoritmo e da análise de complexidade/green computing, pois a avaliação será individual.
- Sugere-se incluir demonstração da aplicação em instâncias de teste e gráfico(s) simples mostrando crescimento do tempo de execução e discussão de impacto energético.

Critérios de avaliação e regras

- **Formação das equipes:** até 3 alunos por equipe (preferencialmente 3).
- **Avaliação individual:** embora o trabalho seja em grupo, a nota será **individual**, baseada no domínio demonstrado na apresentação e na autoria das partes do relatório/código.

Critérios de nota

- Correção da modelagem e da implementação: 30%
- Profundidade da análise de complexidade (teórica + empírica): 20%
- Qualidade da análise de green computing (uso adequado das referências, clareza da relação entre algoritmo e consumo de energia): 15%
- Clareza, organização e qualidade do relatório técnico: 15%
- Apresentação oral e domínio do conteúdo por cada integrante: 20%

Regras gerais:

- Trabalhos com plágio (código ou texto) serão desconsiderados.
- Qualquer uso de código de terceiros deve ser claramente referenciado.
- É permitido o uso de bibliotecas padrão ou frameworks, desde que a parte algorítmica principal seja compreendida e explicada pelos alunos.